

1. Información general

Endpoint QA: <https://dev-api.skyairline.com/partners/v1/user>

Método: POST

Protocolo: HTTPS

Content-Type: application/json

Autenticación: Bearer Token (JWT HS256)

Seguridad adicional: API Management Subscription Key

Encriptación Payload: AES-256-CBC (IV + Ciphertext en HEX)

2. Credenciales de integración (QA)

Tipo	Valor
Secreto compartido (JWT + AES)	9b7c2d4e1a6f8c3d5e7a9b0c1d2e3f4a5c6b7d8e9f0a1b2c3d4e5f6a7b8c9d01
Ocp-Apim-Subscription-Key	5cfc7d2fea014ede83c5bd0086d81d2c
Identificador del Partner	BSPE

⚠ Estas credenciales corresponden únicamente al ambiente QA y deben almacenarse en un gestor seguro de secretos.

3. Headers requeridos

Authorization: Bearer {jwt_token}

Ocp-Apim-Subscription-Key: {Ocp_subscription_key_proporcionada_por_SKY}

Content-Type: application/json

3.1 Payload del JWT

```
{
  "partnerId": "BSPE",
  "iat": <timestamp>,
  "exp": <timestamp>
}
```

El secreto utilizado para firmar el JWT es el mismo utilizado para cifrar el payload.

4. Estructura del Request

```
{
  "partnerId": "BSPE",
  "data": "<encrypted_string_hex>"
}
```

4.1 Estructura interna antes de cifrado

```
{
  "profile": {
    "userId": "email@dominio.com",
    "homeMarket": "PE"
  },
  "document": {
    "type": "DNI",
    "number": "XXXXXXXX",
    "country": "PE"
  },
  "userInfo": {
    "firstName": "Nombre",
    "lastName": "Apellido",
    "emails": ["email@dominio.com"],
    "birthdate": "YYYY-MM-DD"
  }
}
```

5. Respuesta exitosa

HTTP Status real: 201 Created

Nota: El campo httpStatus dentro del body retorna 200 (comportamiento legacy).

```
{
  "success": true,
  "httpStatus": 200,
  "message": "User registered successfully",
  "content": {
    "loginBffResponse": {
      "success": true,
      "httpStatus": 200,

```

```

    "message": "User registered successfully",
    "content": {
      "success": true,
      "response": {
        "uid": "olsMan0e7nAlFGhPrBr9YnlOZvu1",
        "email": "email@dominio.com",
        "emailVerified": false,
        "displayName": "Nombre",
        "disabled": false,
        "metadata": {
          "lastSignInTime": null,
          "creationTime": "Tue, 17 Feb 2026 18:20:21 GMT"
        },
        "tokensValidAfterTime": "Tue, 17 Feb 2026 18:20:21 GMT",
        "providerData": []
      }
    }
  }
}

```

6. Respuestas de Error

400 – Bad request

```

{
  "success": false,
  "httpStatus": 400,
  "message": "Error to register user",
  "content": {
    "error": [
      "homeMarket should not be empty"
    ]
  }
}

```

403 – Partner no autorizado

```

{
  "success": false,
  "httpStatus": 403
}

```

406 – Error de descriptación

```
{
  "success": false,
  "httpStatus": 406,
  "message": "Error: Decryption failed",
  "content": {}
}
```

409 – Documento de identidad o email ya registrado

```
{
  "success": false,
  "httpStatus": 409,
  "message": "Error to register user",
  "content": {
    "error": "Documents already registered or email exists with another document"
  }
}
```

422 – Firma JWT inválida

```
{
  "success": false,
  "httpStatus": 422,
  "message": "JsonWebTokenError: invalid signature",
  "content": {}
}
```

ANEXO A – Script Postman para generación de payload cifrado

```
// Pre-request Script (Postman)
const CryptoJS = require('crypto-js');

const secret = pm.collectionVariables.get('secret-bspe');
const key = CryptoJS.enc.Hex.parse(secret);

const iv = CryptoJS.lib.WordArray.random(16);

const data = {
  profile: {
    userId: "email@dominio.com",
    homeMarket: "PE"
  },
  document: {
    type: "DNI",
    number: "XXXXXXXX",
    country: "PE"
  },
  userInfo: {
    firstName: "Nombre",
    lastName: "Apellido",
    emails: ["email@dominio.com"],
    birthdate: "YYYY-MM-DD"
  }
};

const jsonData = JSON.stringify(data).trim();

const encrypted = CryptoJS.AES.encrypt(jsonData, key, {
  iv: iv,
  mode: CryptoJS.mode.CBC
});

const encryptedData =
  iv.toString(CryptoJS.enc.Hex) +
  encrypted.ciphertext.toString(CryptoJS.enc.Hex);

pm.environment.set('encryptedData', encryptedData);
```